



Reproductor de música

Integrantes:

Benjamín Hernández

Franco Rodríguez

Daniela Soto

Fecha: 2 de Octubre de 2026

Profesora: Jacqueline Aldridge

Ayudante: Milton Hernández

Asignatura: Estructuras de Datos

Índice

1. Introducción	2
2. Desarrollo	3
2.1. Estructuras utilizadas	3
2.2. Algoritmos utilizados	3
2.2.1. Generadores	3
2.2.2. Fisher Yates	3
2.2.3. Algoritmo principal	4
2.2.4. Algoritmo de ordenamiento	4
2.2.5. Algoritmo de Búsqueda	5
2.3. Ejemplo de ejecución del sistema	5
3. Conclusión	9
4. Contribución por estudiante y Declaración de IA	10
4.1. Contribución del equipo	10
4.2. Declaración de IA	10
5. Bibliografía	11

1. Introducción

En este informe se presenta el desarrollo de un simulador de reproductor de música programado en lenguaje C. El programa genera datos de forma automática para simular operaciones como la búsqueda, crea una lista de reproducción y reproduce la canción para finalmente guardarlo en un historial de reproducciones.

El objetivo principal de este proyecto es utilizar y comprender los algoritmos vistos en clases, tales como búsqueda binaria, bubble sort y quick sort. Junto a la manipulación de datos en arreglos de manera eficiente en el almacenamiento y gestión del catalogo musical. Y documentar adecuadamente el código y decisiones de diseño.

2. Desarrollo

2.1. Estructuras utilizadas

Este proyecto se basa únicamente en la estructura de tipo discos, donde incluye toda la información que ofrece cada música:

Listing 1: Estructura discos

```

1 typedef struct
2 {
3     int id; /** ID de la musica */
4     char *titulo; /** titulo de la musica */
5     char *artista; /** artista de la musica */
6     char *album; /** Nombre del album */
7     char *genero; /** Nombre del genero */
8     int duracion_seg; /** Duracion de la musica */
9     int anho; /** Año de lanzamiento de la musica */
10    int n_reproducciones; /** Numero de producciones de la musica */
11 }discos_;

```

Se eligen punteros para los caracteres por un mejor manejo de datos entre ellos.

2.2. Algoritmos utilizados

2.2.1. Generadores

Al iniciar el programa, antes de proporcionar un menú, se ejecuta varias funciones de generación de datos aleatorios para llenar los campos de la estructura Discos.

Hay 3 tipos de generadores:

1. Caracter aleatorio: Genera un carácter alfabético (A-Z).
2. Strings seleccionados de un arreglo: Selecciona de forma aleatoria un elemento dentro de un arreglo predefinido de cadenas (como nombres de artistas o géneros) para poblar los metadatos del disco.
3. Entero aleatorio: Genera un valor dentro de un rango específico (ej. entre 1900 y 2026) para establecer el año de edición o el número de canciones.

Existen un total de ochos funciones, debido a que hay ochos campos en la estructura Discos.

2.2.2. Fisher Yates

Una vez generados los datos se utiliza un algoritmo llamado Fisher-yates que permite mezclar aleatoriamente los registros.

Listing 2: fisher-yates

```

1 void fisher_yates(discos_ discos[MAX_DISCOS], int n_discos)
2 {
3     int j = 0;
4     for (int i = n_discos - 1; i > 0; i--){
5         j = rand() % (i + 1);
6
7         discos_ temp = discos[i];
8         discos[i] = discos[j];
9         discos[j] = temp;
10    }
11 }

```

Se utiliza un ciclo for que recorre el arreglo desde el ultimo elemento y va decreciendo hasta llegar al segundo elemento, y mediante el código `j = rand() % (i + 1)` se obtiene un índice aleatorio dentro del rango de elementos aun no intercambiados. Para hacer el intercambio de estructuras se utiliza una variable temporal `temp` de tipo `discos` para realizar el intercambio de posiciones.

Esta versión de Fisher-Yates es la más moderna (desarrollado por Richard Durstenfeld) que a comparación de su versión original este algoritmo hace el intercambio en el mismo arreglo, mientras que su versión anterior los pasaba a un segundo arreglo, lo que requeriría duplicar su memoria.

2.2.3. Algoritmo principal

Al empezar el programa, si es la primera vez que lo ejecutas. Se generan datos aleatorios para los campos de los discos generados. La cantidad de discos generados de igual manera es aleatoria.

Al empezar el programa por segundo vez, siendo que los datos fueron generados. Se importaran los datos generados por primera vez, tomando en referencia el archivo CSV generado al finalizar el programa correctamente.

Aquí sucede el loop completo del programa. La función Menu() guía al usuario para manejarse en la zona de comandos y poder acceder a todas las funciones que ofrece.

Se imprime por pantalla las siguientes funcionalidades del programa:

1. Salir del programa: Finaliza la ejecución del sistema.
2. Búsqueda por ID: Localiza un disco por su identificador único
3. Búsqueda por Artista: Muestra todas las canciones del artista elegido.
4. Búsqueda por Género Musical: Muestra todas canciones de un correspondiente genero musical
5. Fila de reproducción: Muestra la fila de canciones pendientes. En esta función puede añadir/quitar canciones.
6. Historial de reproducción: Muestra la fila de canciones reproducidas.
7. Reproducir música: Simula el avance de la fila de reproducción.
8. Ver música disponible: Muestra el catalogo completo cargado en memoria.
9. Ranking: Presenta la canciones más escuchadas por artista y genero.
10. Ordenar música disponible: Aplica un algoritmo de ordenamiento según el campo de la estructura Disco que elijas.

2.2.4. Algoritmo de ordenamiento

Este algoritmo es el bubble sort que consiste en recorrer repetidamente el arreglo y comparar los elementos cercanos e intercambiarlos si esta en el orden incorrecto. Este algoritmo se utiliza en la búsqueda por id, donde primero se ordenan de menor a mayor para posteriormente realizar la búsqueda.

Listing 3: Funcion de ordenamiento

```

1 void ordenar_discos_por_id(discos_ discos[MAX_DISCOS], int n_discos) {
2     int swapped;
3     for (int i = 0; i < n_discos - 1; i++)
4     {
5         swapped = 0;
6         for (int j = 0; j < n_discos - i - 1; j++)
7         {
8             if (discos[j].id > discos[j + 1].id)
9             {
10                discos_temp = discos[j];
11                discos[j] = discos[j + 1];
12                discos[j + 1] = discos_temp;
13                swapped = 1;
14            }
15        }
16        if (!swapped)
17        {
18            break;

```

```

19     }
20 }
21 }

```

Quick Sort (Ordenamiento Multicriterio): Se utiliza para la opción general de ordenamiento del menú. Permitiendo ordena eficientemente el catalogo según su criterio elegido por el usuario por (Título, Artista, Álbum, Género, Año o por numero de Producciones) Apoyándose en la función auxiliar `/códigoComprarDiscos`

```

case 2: // Título return strcmp(discos1.titulo, discos2.titulo);
case 3: // Artista return strcmp(discos1.artista, discos2.artista);
case 4: // Álbum return strcmp(discos1.album, discos2.album);
case 5: // Género return strcmp(discos1.genero, discos2.genero);
case 6: // Año return discos1.anho - discos2.anho;
case 7: // Número de reproducciones return discos1.n_reproducciones - discos2.n_reproducciones;
default: return 0;

```

2.2.5. Algoritmo de Búsqueda

Para buscar elementos de tipo entero entre los campos de los discos utilizamos el algoritmo de búsqueda binaria en su forma recursiva.

Se utiliza para buscar exclusivamente el id de una canción introducida por el usuario.

Listing 4: Búsqueda binaria

```

1 int busqueda_binaria_recursiva(discos_ discos[], int busqueda, int izquierda, int derecha)
2 {
3     if (izquierda > derecha)
4     {
5         return -1;
6     }
7
8     int indice_mitad = izquierda + (derecha - izquierda) / 2;
9     int Mitad = discos[indice_mitad].id;
10
11     if (busqueda == Mitad)
12     {
13         return indice_mitad; // Retorna el indice
14     }
15
16     if (busqueda < Mitad)
17     {
18         return busqueda_binaria_recursiva(discos, busqueda, izquierda, indice_mitad - 1);
19     }
20     else
21     {
22         return busqueda_binaria_recursiva(discos, busqueda, indice_mitad + 1, derecha);
23     }
24 }

```

2.3. Ejemplo de ejecución del sistema

A continuación se ilustra el funcionamiento del sistema mediante capturas de consola y el flujo de la cola de reproducción:

```

=====
| ARTISTAS DISPONIBLES          |
=====
| Cris MJ                      |

```

```
| Anuel AA |
| Jere Klein |
| Lucky Brown |
| Bad Bunny |
| Katteyes |
| Blessd |
| Kidd Voodoo |
| Feid |
=====
```

```
=====
|Busqueda de canciones |
=====
| 0- Salir del programa |
| 1- Busqueda por id |
| 2- Busqueda por Nombre |
| 3- Busqueda por Artista |
| 4- Busqueda por Genero Musical |
| 5- Fila de reproduccion |
| 6- Historial de reproduccion |
| 7- Reproducir musica |
| 8- Ver musica disponible |
| 9- Ranking |
| 10- Ordenar musica disponible |
=====
```

8

```
=====
| ID | TITULO | ARTISTA | ALBUM |
=====
| 1011 | Deprisa Pan terrorista | Cris MJ | Sencillo |
| 1006 | Aqui Bordoli Desgraciado | Anuel AA | Canada sangriento |
| 1004 | Acaso Canada Sangriento | Jere Klein | Edgardo estancado |
| 1003 | Aqui Canada terrorista | Lucky Brown | Sencillo |
| 1013 | Acaso Canada Montanhista | Bad Bunny | Sencillo |
| 1007 | Deprisa Canada Estancado | Katteyes | Sencillo |
| 1005 | Aqui Torre Desgraciado | Blessd | Pan estancado |
| 1008 | Pronto Torre Montanhista | Blessd | Torre sangriento |
| 1002 | Aqui Pan Sangriento | Katteyes | Sencillo |
| 1000 | Aqui Pan terrorista | Kidd Voodoo | Canada montanhista |
| 1009 | Deprisa Edgardo Estancado | Feid | Bordoli sangriento |
| 1012 | Deprisa Torre Estancado | Feid | Sencillo |
| 1001 | Pronto Edgardo Desgraciado | Bad Bunny | Edgardo terrorista |
| 1010 | Jamas Pan Estancado | Anuel AA | Canada estancado |
=====
```

```
=====
|Busqueda de canciones |
=====
| 0- Salir del programa |
| 1- Busqueda por id |
| 2- Busqueda por Nombre |
```

```
| 3- Busqueda por Artista |
| 4- Busqueda por Genero Musical |
| 5- Fila de reproduccion |
| 6- Historial de reproduccion |
| 7- Reproducir musica |
| 8- Ver musica disponible |
| 9- Ranking |
| 10- Ordenar musica disponible |
=====
```

5

```
=====
|Menu fila de reproduccion |
=====
| 0- Volver al menu |
| 1- Consultar fila de reproduccion |
| 2- Anhadir canccion |
| 3- Quitar cancion |
| 4- Vaciar lista de reproduccion |
=====
```

Elija su opcion: 2

Ingrese el ID de la cancion que desea anhadir discos_1 la fila de reproduccion

ID: 1000

SE ANHADIO A LA FILA DE REPRODUCCION

```
=====
|Menu fila de reproduccion |
=====
| 0- Volver al menu |
| 1- Consultar fila de reproduccion |
| 2- Anhadir canccion |
| 3- Quitar cancion |
| 4- Vaciar lista de reproduccion |
=====
```

Elija su opcion: 0

```
=====
|Busqueda de canciones |
=====
| 0- Salir del programa |
| 1- Busqueda por id |
| 2- Busqueda por Nombre |
| 3- Busqueda por Artista |
| 4- Busqueda por Genero Musical |
| 5- Fila de reproduccion |
| 6- Historial de reproduccion |
| 7- Reproducir musica |
| 8- Ver musica disponible |
| 9- Ranking |
| 10- Ordenar musica disponible |
=====
```


7

```
=====
|                                     REPRODUCCIENDO CANCION                                     |
=====
| 1000 | Aqui Pan terrorista           | Kidd Voodoo       | Canada montanhista |
=====
```

```
=====
|Busqueda de canciones           |
=====
| 0- Salir del programa         |
| 1- Busqueda por id            |
| 2- Busqueda por Nombre        |
| 3- Busqueda por Artista        |
| 4- Busqueda por Genero Musical |
| 5- Fila de reproduccion        |
| 6- Historial de reproduccion   |
| 7- Reproducir musica           |
| 8- Ver musica disponible       |
| 9- Ranking                     |
| 10- Ordenar musica disponible  |
=====
```

6

```
=====
|                                     HISTORIAL                                     |
=====
| ID   | TITULO                               | ARTISTA           | ALBUM               |
=====
| 1000 | Aqui Pan terrorista                 | Kidd Voodoo       | Canada montanhista |
=====
```

```
=====
|Busqueda de canciones           |
=====
| 0- Salir del programa         |
| 1- Busqueda por id            |
| 2- Busqueda por Nombre        |
| 3- Busqueda por Artista        |
| 4- Busqueda por Genero Musical |
| 5- Fila de reproduccion        |
| 6- Historial de reproduccion   |
| 7- Reproducir musica           |
| 8- Ver musica disponible       |
| 9- Ranking                     |
| 10- Ordenar musica disponible  |
=====
```

0

Datos guardados correctamente

3. Conclusión

En este proyecto se realizó un simulador de reproductor de música en lenguaje C. A través de este desarrollo se logró consolidar los conocimientos antes vistos en clase para poder aplicarlos en la práctica, donde se destacan los siguientes. Se implementaron dos algoritmos de ordenamiento, el primero Bubble Sort para mejorar las búsquedas por ID y el segundo algoritmo Quick Sort para el ordenamiento de multicriterio y por otra parte se utilizó un algoritmo de búsqueda binaria en su forma recursiva. Se integró el uso de CSV para su lectura y escritura, con el fin de guardar los datos entre las ejecuciones.

4. Contribución por estudiante y Declaración de IA

4.1. Contribución del equipo

- **Benjamín Hernández:** QuickSort, Búsqueda Binaria, Insertion Sort para ranking, búsqueda por artista e importación/exportación de archivos CSV.
- **Franco Rodríguez:** Menú principal, generadores aleatorios, módulo de reproducción e historial, búsqueda por género y liberación de memoria dinámica.
- **Daniela Soto:** Generadores aleatorios, gestión completa de la Fila de Reproducción, lógica de desplazamiento de elementos y fisher-yates.

4.2. Declaración de IA

- **Benjamín Hernández:** Se declara el uso de Inteligencia Artificial como apoyo complementario en las siguientes partes del proyecto:
 - Depuración de código en C.
 - Diagnostico y solución del error lógico en la lectura de archivos CSV
 - Corrección de avisos de compilación de los *Warnings* en la ejecución de make
- **Franco Rodríguez:** Uso de de herramientas de Inteligencia Artificial para preguntas conceptuales y búsqueda de errores a la hora de proponer solución en partes del proyecto
 - Comparación de strings
 - Uso de memoria de Punteros
 - Depuración de código en C
- **Daniela Soto:** Se declara uso de herramienta de Inteligencia Artificial para los siguientes puntos:
 - Depuración de codigo en C
 - Correccion de avisos de compilacion de *Warnings*
 - Paso de parámetros por punteros

5. Bibliografía

Referencias

- [1] Tutorialspoint. *DSA Fisher-Yates Shuffle Algorithm*. Disponible en: https://www-tutorialspoint-com.translate.goog/data_structures_algorithms/dsa_fisher_yates_shuffle_algorithm.htm?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc